Lab Report — Lab 04

ANN & Deep Learning

Heart Disease Classification using
Neural Network and AdaBoost

Name	Muhammad Abdullah
Roll Number	BSCS23070
Course	ANN & Deep Learning
Lab	Lab 04
Dataset	UCI Heart Disease (Cleveland, 303 instances)
Submission	Lab Report

This report documents the implementation and analysis of a binary heart disease classifier built in two ways: a custom feedforward neural network (PyTorch) and an AdaBoost ensemble. All steps follow the lab handout, from data loading and preprocessing through training, evaluation, and final comparison.

---

1. Dataset Overview

The UCI Cleveland Heart Disease dataset was used for this lab. It contains 303 patient records across 13 clinical features and one target column. The original target ranges from 0 to 4, but was converted to a binary label: 0 (no disease) and 1 (disease present). Two columns — **ca** and **thal** — contain a small number of missing values, handled during preprocessing.

Feature	Type	Description
age	Numeric	Age in years
sex	Categorical	0 = female, 1 = male
cp	Categorical	Chest pain type (1–4)

trestbps	Numeric	Resting blood pressure (mm Hg)
chol	Numeric	Serum cholesterol (mg/dl)
fbs	Categorical	Fasting blood sugar > 120 mg/dl (0/1)
restecg	Categorical	Resting ECG results (0–2)
thalach	Numeric	Max heart rate achieved
exang	Categorical	Exercise-induced angina (0/1)
oldpeak	Numeric	ST depression induced by exercise
slope	Categorical	Slope of peak exercise ST segment (1–3)
ca	Categorical	Major vessels coloured by fluoroscopy (0–3)
thal	Categorical	Thalassemia type (3/6/7)
target	Binary	0 = no disease, 1 = disease (derived)

---

2. Environment Setup

Google Colab was used as the runtime. The following libraries were imported:

```
import pandas as pd, numpy as np import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split from sklearn.preprocessing
import StandardScaler, OneHotEncoder from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline from sklearn.impute import SimpleImputer from
sklearn.ensemble import AdaBoostClassifier from sklearn.metrics import
accuracy_score, precision_score, recall_score, f1_score import torch, torch.nn as
nn from torch.utils.data import TensorDataset, DataLoader np.random.seed(42);
torch.manual_seed(42)
```

Note: Reproducibility seeds were set for both NumPy and PyTorch before any model operations.

---

3. Data Loading and Exploration

Loading the dataset

The file has no header row and uses **?** as the missing-value marker. Column names were assigned manually and the target was binarised immediately after loading.

```
df = pd.read_csv(DATA_PATH, header=None, names=column_names, na_values='?')
df['target'] = (df['target'] > 0).astype(int) print(df.shape)
```

```
Output: (303, 14)
```

Basic exploration (Cell 4)

Samples	303

Features (after target sep.)	13
All columns stored as	float64 (except target: int64)
Target: class 0 (no disease)	164 samples (54.1%)
Target: class 1 (disease)	139 samples (45.9%)

4. Class Balance Check

Class 0 holds 164 samples (54%) and class 1 holds 139 (46%). The gap is small — about 8 percentage points — so the dataset is roughly balanced. Standard accuracy is a meaningful metric here and no resampling was needed.

Dataset is roughly balanced: Class 0 = 164 (54.1%) | Class 1 = 139 (45.9%)

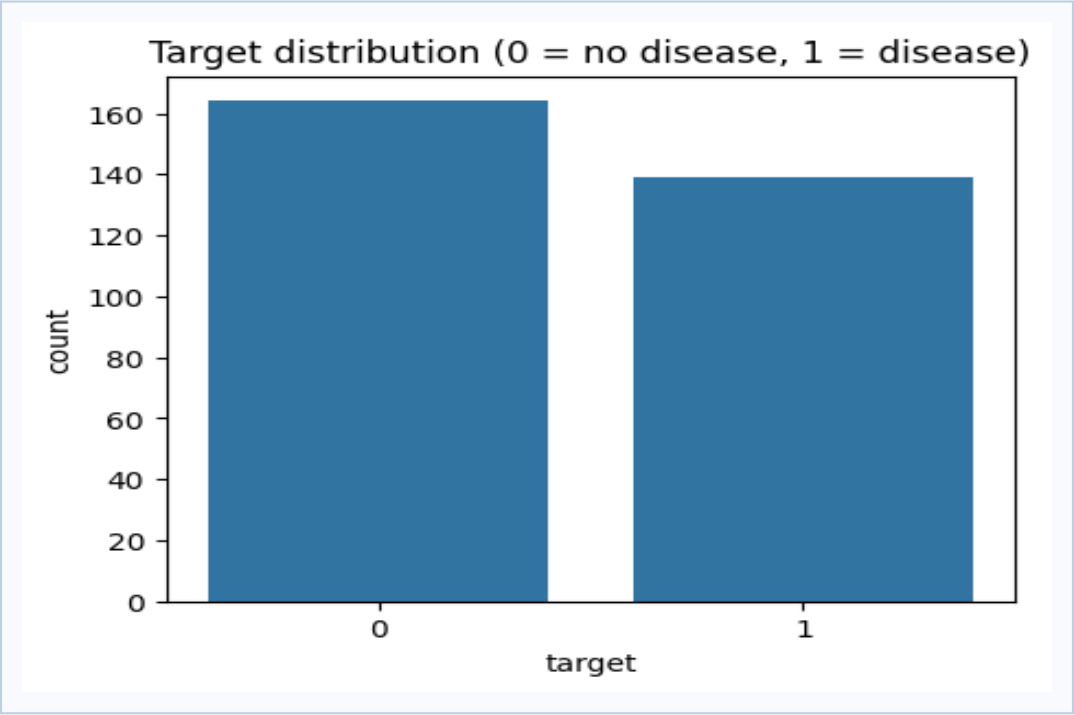


Figure 1 — Target distribution (0 = no disease, 1 = disease). Both bars are close in height, confirming near-balanced classes.

5. Data Quality Assessment

Cell 6 checked missing values, duplicates, and unique counts per column.

Column	Missing	Unique Values	Action

ca	4	4	Impute with most frequent value
thal	2	3	Impute with most frequent value
All others	0	2–152	No action needed

There are no duplicate rows. All imputation was applied inside a Pipeline fitted on training data only, so no test-set information leaked into the imputer statistics.

---

6. Data Visualizations

Four plots were produced in Cell 7. Each is shown below with a brief interpretation.

Figure 2 — Target Distribution

Two bars of nearly equal height confirm the class balance already measured in Cell 5.

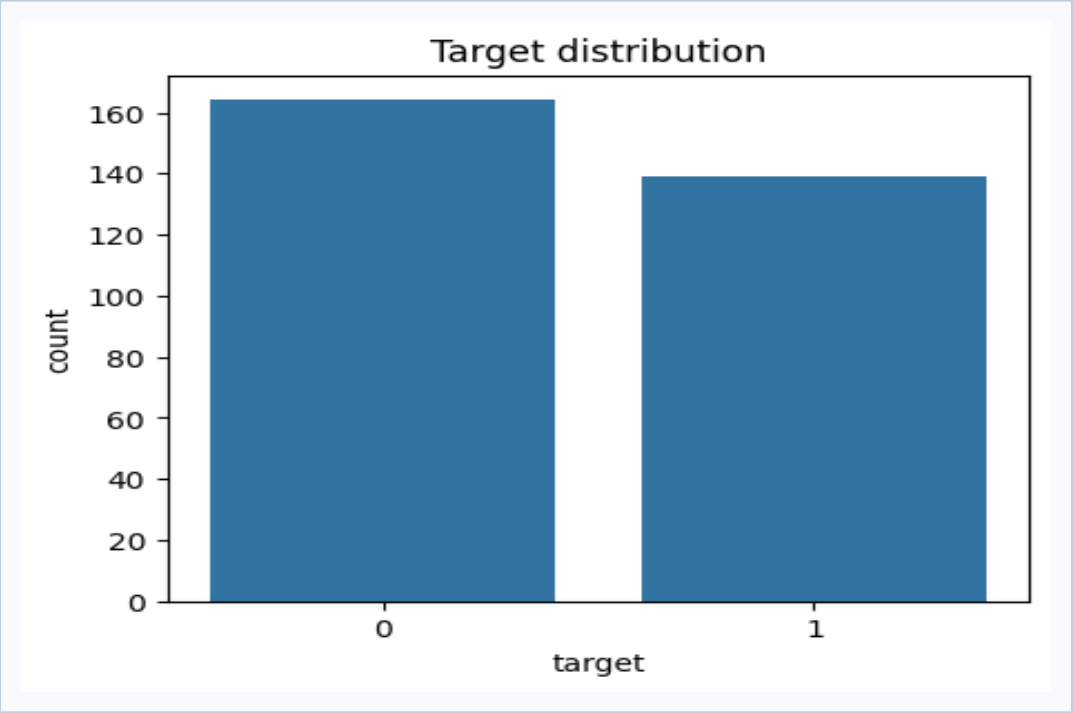


Figure 2 — Target distribution (count plot). Class 0: 164, Class 1: 139.

Figure 3 — Age Distribution

Ages concentrate between 45 and 65 years. The distribution is slightly left-skewed, meaning younger patients are less common. No extreme outliers were visible.

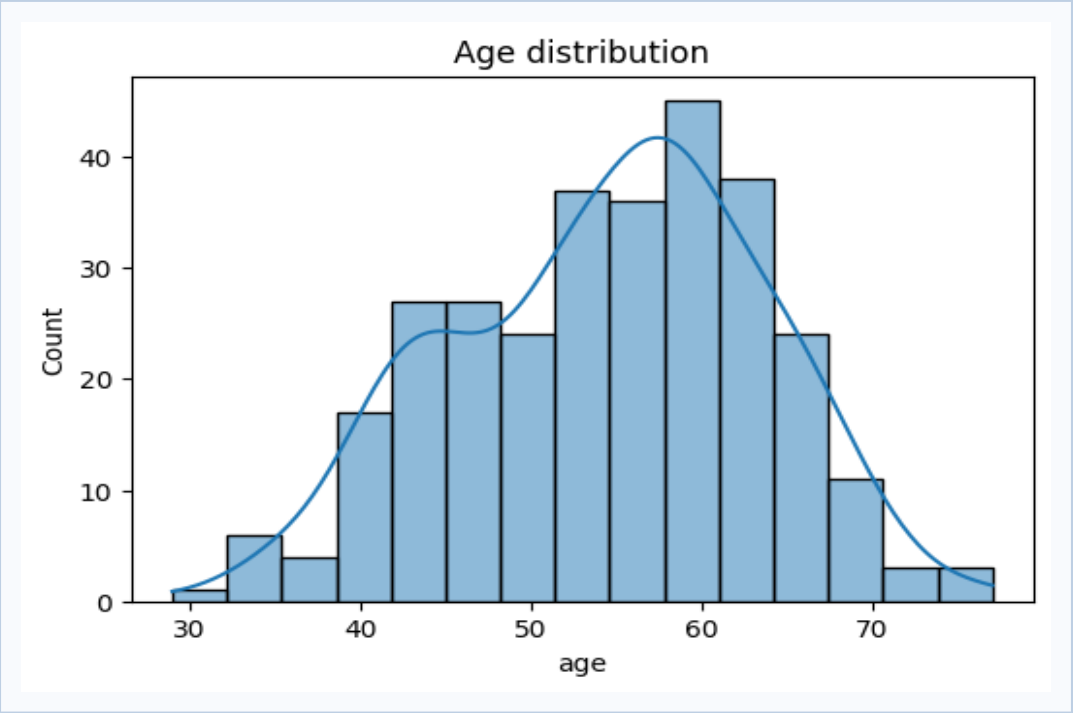


Figure 3 — Age distribution with KDE. Most patients fall in the 45–65 age bracket.

Figure 4 — Max Heart Rate vs Target (thalach)

Patients with heart disease (target = 1) have a noticeably lower median max heart rate than those without disease. This makes thalach one of the more discriminative features.

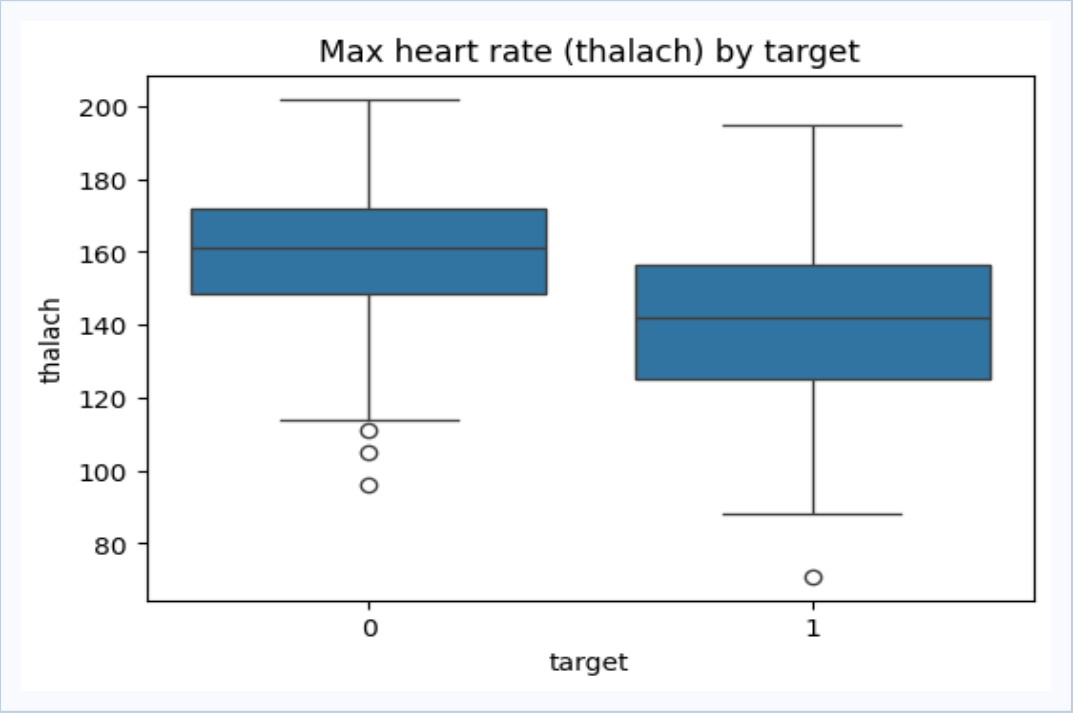


Figure 4 — Boxplot of maximum heart rate (thalach) by target class. Disease patients show lower thalach values.

Figure 5 — Correlation Heatmap

oldpeak has the strongest positive correlation with the target (+0.42). thalach has the strongest negative correlation (–0.42). age and trestbps show weak positive links. chol has almost no linear

relationship with the target.

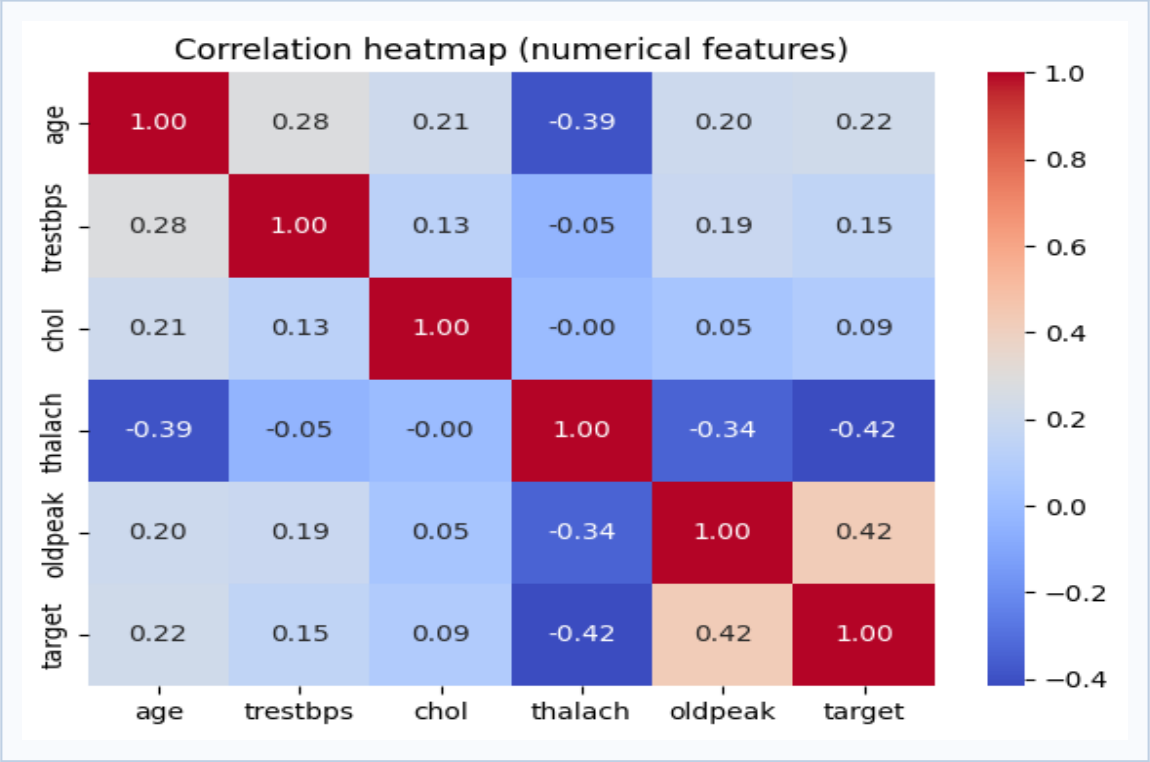


Figure 5 — Correlation heatmap of numerical features. oldpeak and thalach are the most informative numeric predictors.

---

7. Preprocessing Pipeline

Preprocessing followed a pipeline-first approach to prevent data leakage. The data was split before any fitting happened.

Train / Validation / Test Split

Split	Size	% of Total	Purpose
`Train`	217	71.6%	Fit model weights
`Validation`	25	8.2%	Monitor val loss during training
`Test`	61	20.0%	Final evaluation (held out throughout)

Pipeline structure

- **Numeric columns** (age, trestbps, chol, thalach, oldpeak): median imputation then StandardScaler.
- **Categorical columns** (sex, cp, fbs, restecg, exang, slope, ca, thal): most-frequent imputation then OneHotEncoder.

After one-hot encoding, the feature count expanded from 13 to **28**. The pipeline was fit only on the training set, then applied with transform() to validation and test.

```
Processed feature count (input_dim): 28
```

---

8. Neural Network Architecture

Architecture table (Cell 9)

A compact network was chosen given that the training set is only 217 samples. Wider or deeper networks would overfit quickly.

Layer	Input Units	Output Units	Activation
Input	28 (input_dim)	—	—
Hidden 1	28	32	ReLU
Hidden 2	32	16	ReLU
Hidden 3	16	8	ReLU
Output	8	1	none (raw logit)

Manual parameter count (Cell 10)

Layer	Calculation	Parameters
Hidden 1	(28 × 32) + 32	928
Hidden 2	(32 × 16) + 16	528
Hidden 3	(16 × 8) + 8	136
Output	(8 × 1) + 1	9
TOTAL	928 + 528 + 136 + 9	1,601

```
PyTorch verified: Trainable parameters = 1601 ✓
```

---

9. Training Configuration

Loss function — BCEWithLogitsLoss

Binary Cross-Entropy with Logits was chosen because the task is binary and the output layer produces a single raw logit. This combines sigmoid + BCE in a numerically stable way.

Optimizer — Adam (lr = 0.001)

Adam was selected because it adapts the learning rate per parameter and generally converges faster than plain SGD with less manual tuning. Default betas (0.9, 0.999) were kept.

Epochs (initial run)	80

Batch size	32
Optimizer	Adam
Learning rate	0.001
Loss	BCEWithLogitsLoss
Validation split	25 samples (held-out subset)

---

10. Training Progress and Overfitting Analysis

Loss values at selected epochs (80-epoch run)

Epoch	Train Loss	Val Loss	Observation
1	0.6930	0.6847	Both high — random-chance level
10	0.5778	0.5549	Both falling together
20	0.3459	0.3196	Good generalisation
30	0.3075	0.3133	Val loss nearly matched train
40	0.2811	0.2960	Val loss begins to diverge
50	0.2560	0.3262	Overfitting now visible
60	0.2366	0.3330	Gap widening
70	0.2104	0.4026	Clear overfitting
80	0.1802	0.4117	Final — val loss is 2× train loss

The training and validation losses track closely up to about epoch 40. After that, train loss keeps falling while validation loss climbs back up. By the end of 80 epochs the gap is large — 0.18 vs 0.41.

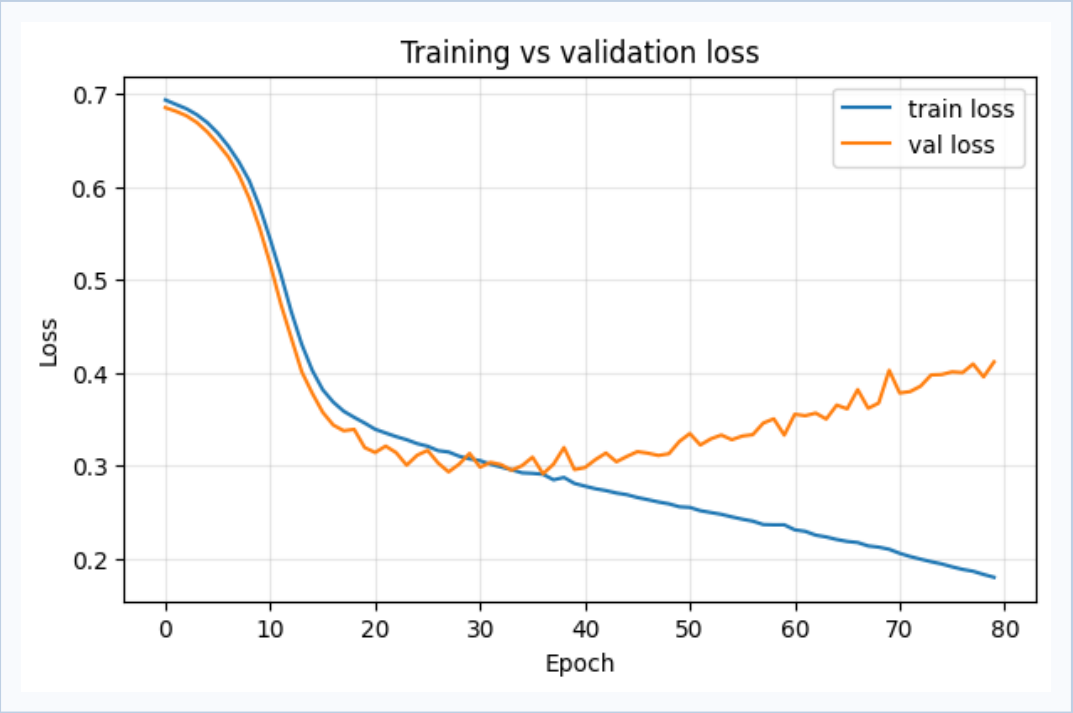


Figure 6 — Training vs validation loss over 80 epochs. Val loss starts rising after epoch ~40, indicating overfitting.

---

11. Early Stopping Experiment

Because the 80-epoch run showed overfitting after epoch 40, the network was re-trained from scratch with early stopping: training halted when validation loss had not improved for 10 consecutive epochs, and the best weights were restored.

Patience	10 epochs
Stopped at	Epoch 47
Best epoch	Epoch 37
Best val loss	0.2914

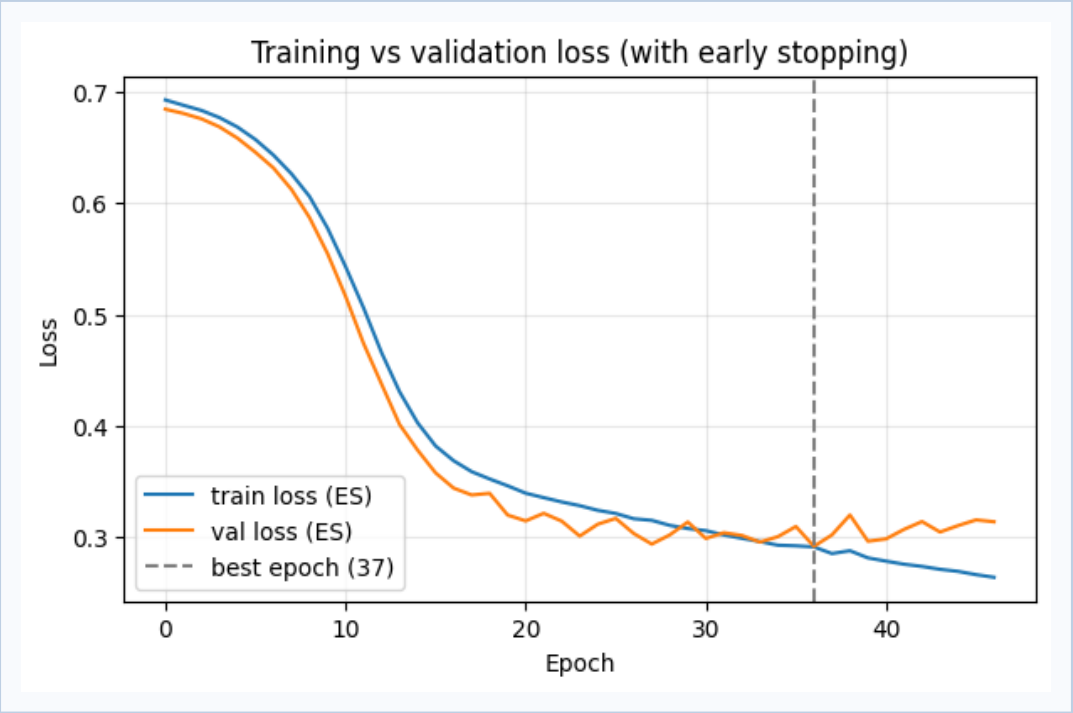


Figure 7 — Training vs validation loss with early stopping. The dashed grey line marks the best epoch (37). Training stopped at epoch 47.

The early-stopped model captures the point where the network generalises best before it starts memorising noise in the training data.

---

12. Model Evaluation Results

Neural Network (80 epochs, original run)

Metric	Value
Accuracy	0.8852 (88.52%)
Precision	0.8621
Recall	0.8929
F1-Score	0.8772

Neural Network with Early Stopping (best at epoch 37)

Metric	Value
Accuracy	0.8852 (88.52%)
Precision	0.8387 (lower — more false positives)
Recall	0.9286 (higher — catches more disease cases)
F1-Score	0.8814 (slightly improved)

AdaBoost (50 estimators)

```
ada_model = AdaBoostClassifier(n_estimators=50, random_state=42)
ada_model.fit(X_train_p, y_train.values) y_pred_ada = ada_model.predict(X_test_p)
```

Metric	Value
Accuracy	0.8852 (88.52%)
Precision	0.8621
Recall	0.8929
F1-Score	0.8772

---

13. Final Model Comparison

Model	Accuracy	Precision	Recall	F1-Score
Neural Net (80 epochs)	0.8852	0.8621	0.8929	0.8772
Neural Net + Early Stopping	0.8852	0.8387	0.9286	0.8814
AdaBoost (50 estimators)	0.8852	0.8621	0.8929	0.8772

All three setups achieve the same test accuracy of **88.52%**. The original neural net and AdaBoost match on all four metrics. The early-stopping variant trades a small amount of precision for a notable gain in recall — important in a medical setting where missing a true disease case is costly.

---

14. Discussion and Conclusions

- **Class balance:** With 54% / 46% split, no resampling was needed. Standard accuracy is meaningful here.
- **Missing values:** Only ca (4 missing) and thal (2 missing) required imputation inside a fitted pipeline — no leakage.
- **Feature expansion:** One-hot encoding grew the feature space from 13 to 28 columns. A small network (1,601 parameters) was sufficient.
- **Overfitting:** The 80-epoch run began overfitting after epoch 40. Validation loss rose from 0.29 to 0.41 by the final epoch (Figure 6).
- **Early stopping:** Stopping at epoch 47 (best weights from epoch 37, val loss 0.2914) prevented the late-stage overfit. Recall improved from 0.89 to 0.93 (Figure 7).
- **AdaBoost vs Neural Net:** Both matched on every metric. AdaBoost is simpler and needs no training loop. The neural net is more flexible but requires care on training length.
- **Next steps:** Dropout regularisation, hyperparameter search over n_estimators for AdaBoost, or a larger dataset could differentiate the models further.

Submitted by: Muhammad Abdullah | Roll No. BSCS23070